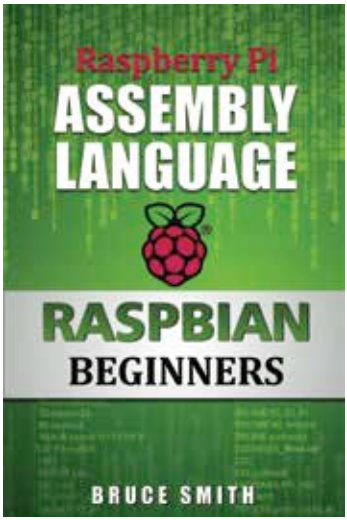
by computers. Every line of command written in assembly code goes directly to the computer's processor for execution. This is why at its very basic level all these commands are actually very simple in design. Only when we see them all together in millions of line do we feel that it is an alien language.



Learning assembly language is a must for advanced projects.

It is important to know that different processors require different assembly code languages. So the assembly code known for an Intel processor won't work on an ARM processor and vice versa. However, thanks to an operating system we are able to sidestep the hassle of rewriting assembly code for each processor and are able to have it executed and translated via the operating system to its required version. Thankfully, since most operating systems are written in C or C++, with assembly code used only in the most necessary sections.

Baking Your Pi - OS Development

The process of writing your own operating system on the Raspberry Pi is called "baking". Typically, the Pi is used with a pre-made operating system like Raspbian or Pidora, however, if your project requires a set of custom instructions for the hardware, as in the case of robotics, game emulation or internet relays, then a baked operating system is best. As of now there are hundreds of thousands of different baked operating systems available online which can accomplish many project goals. We will be discussing the process by which you can write a very simple operating system with one simple purpose - to activate a LED light on the Raspberry Pi. The LED is located near the RCA and USB port. This activation operation is known as the "OK" or "ACT" command for the LED because the light was originally labelled as OK and then changed to ACT in the revised boards.

Once you have visited the URL link provided early in the chapter and downloaded the GNU Toolchain and OS Templates we are ready to begin.